Automatisation du développement

Points de la semaine

- Commande Makers
- CI (au travers des Github Actions)

Les Makers

Les Makers

- Noms différents selon les frameworks
- Outils qui permettent de générer du code

Que générer ?

- Controller
- Entity
- Form
- Command
- ETC.

Pourquoi les utiliser ?

- Gain de temps
- Modifier la BDD simplement ("migration")
- Uniformisation du code
- Réduction des erreurs

Les Makers

- Symfony : `php bin/console make:controller`
- Laravel : `php artisan make:controller`
- Résultat différent selon les frameworks

Résultat

```
I have no name!@97508f2e8b8b:/var/www/html$ php bin/console doctrine:fixtures:load

 Careful, database "main" will be purged. Do you want to continue? (yes/no) [no]:
 > y

   > purging database
   > loading App\DataFixtures\FilmsFixtures
I have no name!@97508f2e8b8b:/var/www/html$ php bin/console make:controller

 Choose a name for your controller class (e.g. GentlePopsicleController):
 > Test

 Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
 > n

 created: src/Controller/TestController.php
 created: templates/test/index.html.twig

 Success!

 Next: Open your new controller class and add some pages!
```

Code généré

TestController.php

```php
<?php

namespace App\Controller;

use ...

final class TestController extends AbstractController  no usages
{
    #[Route('/test', name: 'app_test')]
    public function index(): Response
    {
```

Bonnes pratiques

- Relire le code généré
- Personnaliser selon les besoins
- Ne pas « tout générer »
- Versionner les changements

CI/CD

CI/CD ?

- Continuous Integration => actions automatiques quand on push
- Continuous Delivery => déploiements automatiques une fois validé

CI/CD - Le but de la CI

- Automatisation des tests (code stable, erreurs remontées rapidement)
- Sur un serveur dédié (pas de dépendance aux développeurs)
- Se lance à chaque action sur le projet (push, merge)

CI/CD - Exemple de services

- Github Actions
- Gitlab CI
- Travis Ci
- Circle CI
- Jenkins
- Etc.

CI/CD - Principes généraux

- => toujours le même principe : pipeline automatisé à partir d'un fichier de configuration
- Des déclencheurs
- Des étapes
- Des actions
- Des variables et secrets

Ex. fichier de conf github

```yaml
name: tests
on:
  push:
    branches: [ main ]
jobs:
  php-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup PHP
        continue-on-error: false
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.2'
      - run: composer install --no-progress
      - run: php bin/phpunit
```

Pipeline github

← actions

add github actions #1

Summary

All jobs

symfony-tests

Run details

Usage

Workflow file

symfony-tests

failed now in 10s

Set up job

Run shivammathur/setup-php@v2

```
1  ▸ Run shivammathur/setup-php@v2
6  /usr/bin/bash /home/runner/work/_actions/shivammathur/setup-php/v2/src/scripts/run.sh
7  ==> Setup PHP
```

Run actions/checkout@v4

Copy .env.

Install Dependencies

Execute tests (Unit and Feature tests) via PHPUnit

Post Run actions/checkout@v4

Complete job

Résultat github

Execute tests (Unit and Feature tests) via PHPUnit

```
1  ▸ Run echo "Execute phpunit"
9  Execute phpunit
10 > vendor/bin/phpunit
11 PHPUnit 9.6.22 by Sebastian Bergmann and contributors.
12
13 Testing
14 ..                                                              2 / 2 (100%)
15
16 Time: 00:00.024, Memory: 8.00 MB
17
18 OK (2 tests, 2 assertions)
```

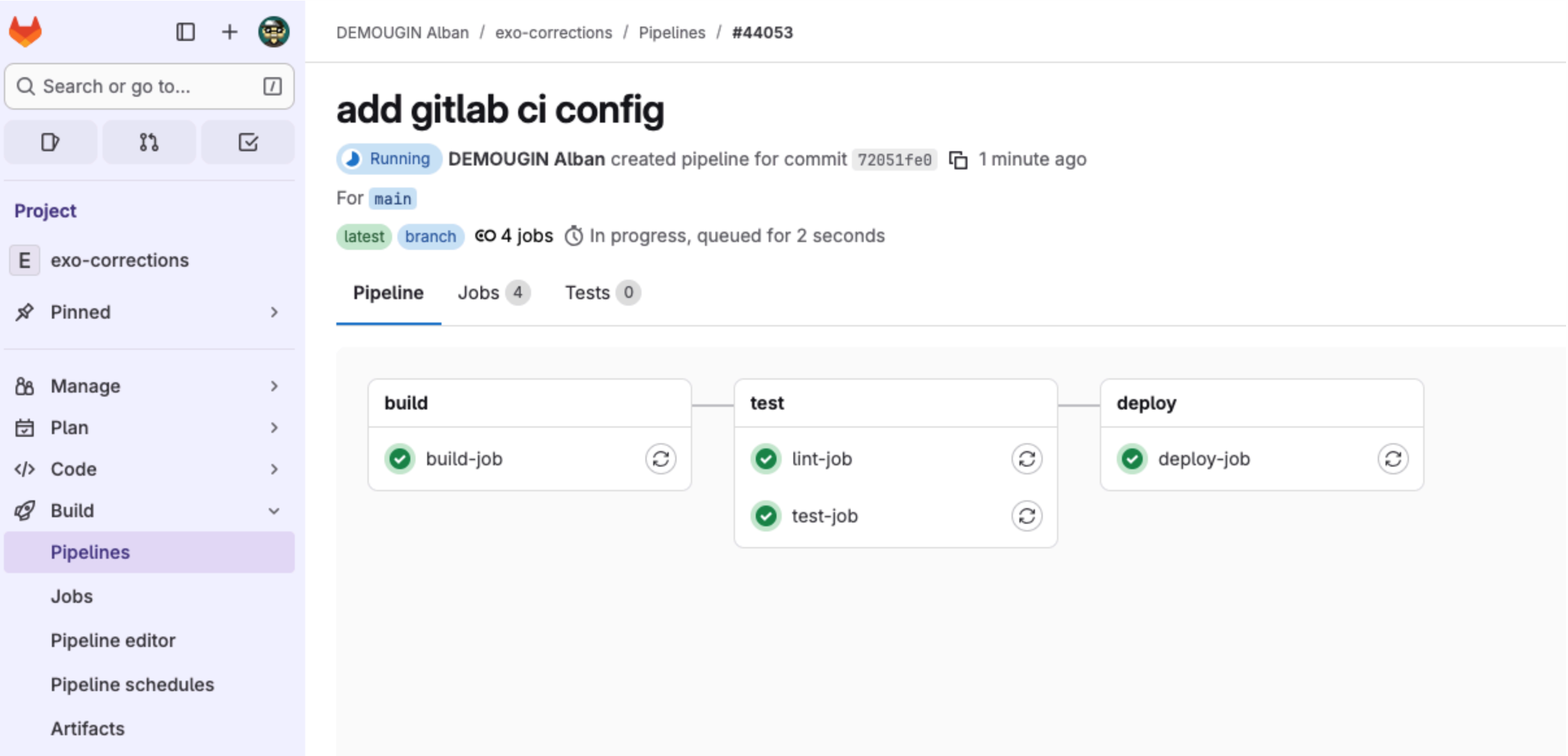
Execute linters via phpcs

```
1  ▸ Run echo "execute phpcs"
9  execute phpcs
10 > ./vendor/bin/phpcs
11
12 FILE: src/Repository/MovieRepository.php
13 ----------------------------------------------------------------------------
14 FOUND 1 ERROR AFFECTING 1 LINE
15 ----------------------------------------------------------------------------
16  12 | ERROR | [x] Opening brace of a class must be on the line after the
17     |       |     definition
18 ----------------------------------------------------------------------------
19 PHPCBF CAN FIX THE 1 MARKED SNIFF VIOLATIONS AUTOMATICALLY
```

Ex. fichier de conf gitlab

```
image: php:8.3
stages:
  - build
  - test
  - deploy
test-job:
  stage: test
  allow_failure: false
  script:
    - curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer
    - composer install --no-interaction --no-progress --prefer-dist --no-scripts
    - composer run test || exit 1;
....
```

Pipeline gitlab



Résultat gitlab

```
450   - Installing symfony/stimulus-bundle (v2.22.1): Extracting archive
451   - Installing symfony/ux-turbo (v2.22.1): Extracting archive
452   - Installing symfony/validator (v7.2.2): Extracting archive
453   - Installing psr/link (2.0.1): Extracting archive
454   - Installing symfony/web-link (v7.2.0): Extracting archive
455   - Installing symfony/twig-bundle (v7.2.0): Extracting archive
456   - Installing symfony/web-profiler-bundle (v7.2.2): Extracting archive
457   - Installing symfony/yaml (v7.2.0): Extracting archive
458   - Installing twig/extra-bundle (v3.18.0): Extracting archive
459 Generating autoload files
460 118 packages you are using are looking for funding.
461 Use the `composer fund` command to find out more!
462 Run composer recipes at any time to see the status of your Symfony recipes.
463 phpstan/extension-installer: Extensions installed
464 > phpstan/phpstan-doctrine: installed
465 > phpstan/phpstan-symfony: installed
466 $ echo "Running unit tests... "
467 Running unit tests...
468 $ composer run test
469 > vendor/bin/phpunit
470 PHPUnit 9.6.22 by Sebastian Bergmann and contributors.
471 Testing
472 ..                                                                  2 / 2 (100%)
473 Time: 00:00.009, Memory: 6.00 MB
474 OK (2 tests, 2 assertions)
475 $ echo "Unit tests completed."
476 Unit tests completed.
```

A votre tour !